

발표 Q&A 모음 — One-SOL DR 프로젝트 (CI/CD 오케스트레이션 파트)

이 파일 목적: 발표 직전 + Q&A 세션 직전에 빠르게 훑기 위한 *단독 Q&A 모음집*. 세 prep 파일 (*rehearsal* / *script-final* / *talking-points*) 의 Q&A 를 토픽별로 통합.

사용법: 발표 시작 직전 톡 훑고, Q&A 세션 직전에 한 번 더. 토픽별로 정렬되어 있어 질문 들어왔을 때 해당 섹션만 머릿속에 떠올리면 됨.

★ 표시 = 받을 가능성 높음 / 받았을 때 임팩트 큰 답변 🧠 표시 = 최종 대본에서 시간 관계상 잘린 detail의 Q&A 백업 (대본에서 안 다뤘으니 청중이 물을 가능성 높음)

목차

0. 본인 파트 — CI/CD 및 오케스트레이션이란? ★ (Q0-A·B·C·D) — 가장 원초적 질문
1. 시스템 / 아키텍처 결정 (Q1-Q2 + Q2-A·B 🧠)
2. 도구 선택 (Q3-Q5 + Q5-A 🧠)
3. Terraform 기여 framing ★ (Q6-Q10)
4. DR 운영 / RTO·RPO (Q11-Q14)
5. DB / Split-brain / Cascade ★★ (Q15-Q18)
6. 시나리오 / 코드 구조 (Q19-Q24 + Q19-A·B + Q21-A 🧠)
7. AI 협업 / 트러블슈팅 (Q25-Q28)
8. 향후 개선 / 모니터링 (Q29-Q32 + Q29-A·B·C·D 🧠)
9. 데이터 정합성 보완 멘트

0. 본인 파트 — CI/CD 및 오케스트레이션이란? ★

이 섹션은 가장 원초적인 질문 대비. "CI/CD가 뭐예요?", "본인 역할이 정확히 뭐였나요?" 같이 발표 톤 잡기 직전에 받을 수 있는 기초 질문을 묶음.

Q0-A. CI/CD가 정확히 뭔가요? ★

"**CI**는 *Continuous Integration*, 한국어로 *지속적 통합*입니다 — 코드 변경을 자주 main에 통합하면서 자동 빌드·테스트로 충돌·버그를 빨리 잡는 과정입니다.

CD는 두 가지 의미로 통용되는데, 둘 다 맞습니다:

- *Continuous Delivery* (지속적 전달) — 통합된 코드를 *언제든 배포 가능한 상태*로 유지, 실제 배포는 사람이 트리거
- *Continuous Deployment* (지속적 배포) — 테스트만 통과하면 *자동으로* 운영 배포

저희 프로젝트는 운영자가 Jenkins UI에서 드롭다운으로 시나리오를 선택하는 구조라 **Continuous Delivery** 쪽에 가깝습니다 — *사람 결정* 단계를 의식적으로 끼워넣은 1-Click DR의 정의와도 맞아떨어집니다."

Q0-B. CI/CD 및 오케스트레이션 파트에서 정확히 무엇을 담당하셨나요? ★★

"다섯 가지입니다.

1. **자동화 흐름 설계** — 단일 Jenkinsfile에서 `parameters { choice() }`로 3 시나리오 분기 구조
2. **Ansible playbook 설계** — `site.yml`(시나리오 3 전체 셋업), `webwas.yml`(시나리오 1 앱 갱신), role 재사용 패턴
3. **Pipeline ↔ Terraform 통합** — tfvars 레이어 분리(pilot-light/dr-active)와 호출 흐름 통합
4. **안전장치 stage 설계** — replication lag check, DB EC2 격리 SQL 3종, smoke test
5. **네트워크 아키텍처 결정** — HAProxy-only Tailscale Subnet Router 패턴

한 문장으로 줄이면: 세 가지 자동화 도구(Jenkins/Ansible/Terraform)를 '1-Click DR'이라는 하나의 흐름으로 묶는 일 이었습니다."

Q0-C. 이 프로젝트는 CI보다 CD·오케스트레이션이 더 비중이 큰 것 같은데 맞나요?

"맞습니다. 일반적인 CI/CD 파이프라인은 코드 push → 자동 빌드·테스트 → 자동 배포 흐름인데, DR 시스템은 거기에 재해 시 인프라를 전환하는 오케스트레이션 영역이 더해집니다. 그래서 파트 이름도 'CI/CD 및 오케스트레이션' 이에요.

저희 시스템의 비중을 굳이 나누면 — CI는 GitHub에 새 jar push 후 빌드 트리거 정도로 비중이 작고, CD(Continuous Delivery)와 인프라 오케스트레이션(Terraform 시나리오 전환, DB 안전장치 SQL, Tailscale 라우팅)이 훨씬 큼니다."

💡 "이 프로젝트가 진짜 CI/CD인가?" 같은 함정 질문에 대비. 정직하게 "CD·오케스트레이션 중심" 으로 답하면 신뢰 ↑

Q0-D. 이 프로젝트에서 CI/CD와 파이프라인 오케스트레이션을 어떤 도구로 구현했나요? ★★

"다섯 가지 도구를 조합했습니다.

도구	역할	어떤 시나리오에서?
Jenkins	전체 흐름 트리거 + 분기 + stage 실행 (오케스트레이터 본체)	모든 시나리오
Ansible	서버 내부 상태 변경 — 앱 배포, OS 설정, 패키지 설치	시나리오 1(앱 갱신), 3(Failback 전체 셋업)
Terraform	AWS 인프라 상태 변경 (IaC) — ALB Listener Rule, ASG scale 등	시나리오 2(Failover), 3(Failback 전환)
Tailscale	온프레임 ↔ AWS 네트워크 통로 — Jenkins가 양쪽에 닿게 해주는 전제 인프라	모든 시나리오의 전제 조건
Git / GitHub	형상관리 — Jenkinsfile, Ansible playbook, Terraform tf 파일, jar 산출물 저장	모든 시나리오의 코드 출처

통합 흐름은 Jenkins가 중심입니다 — 운영자가 Jenkins UI 드롭다운에서 시나리오를 고르면, Jenkinsfile이 해당 시나리오에 맞게 Ansible playbook을 호출하거나 Terraform apply를 호출합니다. 그 호출이 가능하려면 Jenkins가 온프레임과 AWS 양쪽에 네트워크로 닿아야 하는데, 그 통로가 Tailscale입니다.

비유하자면 Jenkins = 지휘자, Ansible/Terraform = 연주자, Tailscale = 무대 입니다."

💡 Q0-B(다섯 가지 역할)와 Q0-D(다섯 가지 도구)는 역할 vs 도구 차이. Q0-B는 내가 한 일, Q0-D는 내가 쓴 도구. 헛갈리지 말 것.

1. 시스템 / 아키텍처 결정

Q1. 왜 Hybrid Cloud였나요? (전부 클라우드로 옮길 수도 있었을 텐데)

"기존 사내 시스템을 그대로 유지하면서, 연속성을 추가로 확보하는 것이 목적이었습니다. 온프레임 환경을 모두 걷어내고 클라우드로 옮기는 것보다, 운영 환경은 그대로 두고 클라우드에 백업 환경을 두는 쪽이 비용과 운영 부담 면에서 합리적이라고 판단했습니다."

Q2. 왜 Pilot Light 전략이었나요?

"비용은 낮게 유지하면서 연속성을 확보한다는 두 가지 장점 때문에 골랐습니다. 평소에는 온프레임에서 정상 운영 하면서, AWS 측에는 최소한의 자원만 대기시켜 두고, 장애가 발생하면 빠르게 깨워서 서비스 연속성을 보장하는 방식입니다."

Q2-A. Pilot Light에서 AWS측에 평상시 어떤 자원이 살아있고, 재해 시 무엇이 깨어나나요?

🗨️ [최종 대본에서 시간 관계상 잘린 부분 — Q&A 백업] "AWS측에 평상시 살아있는 자원은 SpringBoot ASG, Jenkins, HAProxy 세 가지입니다. 핵심은 *SpringBoot ASG*로, 평시에는 scale을 0으로 유지하다가 재해가 발생하면 0→2로 증가해 온프레미스를 대신해 트래픽을 처리합니다. 이게 Pilot Light의 '깨어나는 자원' 부분입니다. *Jenkins*와 *HAProxy*는 평시에도 깨어 있는데, *Jenkins*는 시나리오 1 앱 갱신 / 시나리오 2 Failover 빌드 트리거에 항상 필요하고, *HAProxy*는 평상시 ALB → AWS HAProxy → Tailscale → 온프레임 경로의 중계점이라 켜져 있어야 합니다."

Q2-B. 온프레임과 AWS의 IP 대역은 어떻게 분리되어 있나요?

🗨️ [최종 대본 슬라이드 [6]에서 잘린 detail — Q&A 백업] "온프레임은 192.168.20.0/24, AWS VPC는 10.0.0.0/16입니다. 두 망이 겹치지 않게 설계해서 라우팅 충돌이 없고, 온프레임 HAProxy의 Tailscale Subnet Router가 두 망 사이의 게이트웨이 역할을 합니다. AWS 측에서 Tailscale이 활성화되지 않은 인스턴스 (SpringBoot ASG, RDS 등)도 VPC Route Table의 라우팅 규칙을 통해 HAProxy AWS를 거치면 온프레임에 도달할 수 있는 구조입니다."

2. 도구 선택

Q3. 왜 Jenkins였나요? (GitHub Actions / GitLab CI 같은 옵션도 있었을 텐데)

"두 가지 이유였습니다. 첫째, Hybrid 환경에서 온프레임과 AWS 양쪽 자원에 모두 접근해야 했는데, Jenkins는 자체 호스팅 방식이라 양쪽 네트워크에 직접 닿을 수 있다는 점이 저희 환경에 맞았습니다. 둘째, `parameters { choice() }` 같은 운영자 입력 기반 빌드 트리거가 기본 기능으로 지원되어서, 1-Click DR의 '인간 결정' 단계에 자연스럽게 들어갔습니다."

Q4. 왜 Ansible이었나요? (Shell script로도 가능했을 텐데)

"역등성과 에이전트리스 구조 두 가지입니다. 역등성은 Failover/Failback처럼 같은 코드를 시나리오마다 반복 실행해야 하는 상황에서 신뢰성을 보장합니다. 에이전트리스 구조는 대상 서버에 별도 프로그램 설치가 필요 없어서, 온프레임 서버들과 AWS EC2 양쪽을 같은 도구로 관리하는 부담이 적었습니다."

Q5. 왜 Tailscale이었나요? (Site-to-Site VPN 같은 옵션도 있었을 텐데)

"WireGuard 기반 P2P 연결이라 별도 게이트웨이 장비 없이도 동작하고, 무료 tier가 있어서 학습 환경에서 시작하기에 부담이 적었기 때문입니다. 호스트 수가 곧 비용을 결정하는 구조라서 운영 측면에서 비용 예측이 명확하다는 점도 저희에게 맞았습니다."

Q5-A. HAProxy-only Tailscale 패턴은 처음부터 이 구조로 설계했나요? ★

🗨️ [최종 대본 슬라이드 [40]에서 잘린 origin story — Q&A 백업] "아닙니다. 처음에는 모든 노드에 Tailscale을 까는 mesh 구조였습니다. 시나리오 3 Failback의 Ansible 빌드를 테스트하다가 ProxyJump 트러블슈팅을 마주했고, 그 해결 과정에서 'Subnet Router로 우회하면 어떨까?' 라는 아이디어를 얻었습니다. 적용해보니 10분 만에 풀렸고, 결과적으로 호스트 수도 6→4로 줄어 비용·보안 양쪽 면에서 더 좋은 설계로 이어졌습니다. 트러블슈팅 과정이 더 좋은 아키텍처로 이어진 사례라고 생각합니다."

💡 이 답변은 [40] 슬라이드의 6→4 숫자가 왜 그런 결과가 나왔는지까지 설명해주는 강력한 backup. Q5 다음에 자연스럽게 받을 수 있는 후속 질문.

3. Terraform 기여 framing ★

핵심 framing 한 문장 (외울 것): "Terraform 모듈은 자원 정의서, Pipeline은 호출 시나리오. 제 영역은 후자에 가깝습니다."

Q6. Terraform 모듈은 본인이 직접 작성하셨나요?

"모듈 자체는 팀 + AI 협업으로 작성됐고, 제 기여는 tfvars 레이어와 Pipeline 통합입니다. 코드 라인 수가 아니라 환경 전환 의사결정에 더 가까운 기여입니다."

Q7. Terraform 코드 직접 보여줄 수 있나요?

"Terraform 모듈 코드 자체는 슬라이드 15(Terraform 담당 팀원)에서 다뤘습니다. 제 슬라이드는 그 모듈을 Pipeline 안에서 어떻게 호출하는지 위주로 구성했어요. Jenkinsfile의 `terraform apply -var-file=dr-active.tfvars` 한 줄이 ALB Listener Rule 교체 + ASG 0→2 scale up까지 다 수행합니다. 호출 한 줄이 어떤 자원 변화를 만드느냐가 제 슬라이드의 메시지에요."

Q8. Terraform 부분에서 본인이 직접 결정한 건 뭐예요?

"세 가지가 제 결정입니다. (1) tfvars를 평상시(pilot-light) / Failover(dr-active) 두 파일로 layered하게 두고 시나리오별로 갈아끼우는 전략. (2) `terraform apply` 호출 전후에 RDS lag=0 대기 / DB EC2 격리 / smoke test 같은 안전장치 stage를 Pipeline에 끼우는 흐름 설계. (3) Phase 3 Failback에서 Jenkins가 양쪽 네트워크에 접근 가능한 유일한 노드라는 점을 활용해 RDS 직접 mysqldump하는 경로 설계 — 이건 cascade 정합성 보장의 핵심이에요."

Q9. Terraform 직접 안 짰으면 본인 기여가 뭔가요? (까칠한 톤) ★

"AI 시대의 엔지니어 기여를 어떻게 정의하느냐의 질문이라고 생각합니다. 제 기여는 세 가지예요 — (1) 시스템 흐름 설계, (2) 안전장치 결정, (3) 검증·판단·채택. 모듈 코드는 AI와 팀이 함께 작성했지만, 그걸 어떤 순서로 호출할지·어떤 안전장치를 끼울지·각 단계의 결과를 검증하고 다음 단계로 넘길지를 결정한 건 제 영역입니다. 코드 라인 수가 아니라 의사결정 quality가 제 기여라고 봅니다."

Q10. Terraform 모듈 구조(4 modules)의 장단점 설명해주세요 (깊은 질문)

"모듈 구조 설계는 Terraform 담당 팀원이 깊게 다뤘습니다. 슬라이드 15가 그 영역이에요. 제 영역에서 답할 수 있는 부분은 — 4 모듈로 분리된 덕분에 Pipeline에서 시나리오별로 다른 모듈만 영향 주는 게 가능했다는 점. 예를 들어 Failover 시 networking 모듈은 건드리지 않고 compute의 ALB Listener Rule만 교체하는 식이에요."

💡 "모르는 건 모른다" 정직 + "내 영역에서 알 수 있는 부분만 답". 신뢰 ↑

4. DR 운영 / RTO·RPO

Q11. 1-Click인데 왜 인간이 결정해야 하나요?

"결정까지 자동화하면 일시적 장애에도 Failover가 트리거되어 사용자 불편 + 비용 손해 + split-brain 위험이 생깁니다. 인간 판단으로 그걸 차단하는 게 운영 안정성 측면에서 더 나은 선택이라고 봤습니다."

Q12. Failover 빌드가 항상 성공하나요?

"오히려 lag check 안전장치 덕분에 replication 끊긴 상태에서는 의도적으로 실패합니다. 데이터 손실 방지가 최우선입니다. 안전장치가 작동해서 시스템이 자기 보호한 사례라고 생각합니다."

Q13. RTO가 왜 이렇게 긴가요? (3-4분대)

"이 환경의 measure입니다. 안전장치 — replication lag check, DB 격리 SQL — 가 시간을 일부 소비합니다. 인스턴스 사이즈, 네트워크, 데이터량에 따라 더 짧아질 여지가 있습니다."

Q14. Failback 시간이 Failover보다 더 긴 이유는?

"Failback은 의도된 계획 다운타임입니다. 안전한 원위치 복구를 위해 정합성 검증 + 데이터 동기화에 시간을 씁니다. Failover의 장애 복구 시간과는 의미가 다른 운영적 선택입니다."

5. DB / Split-brain / Cascade ★★

사실 확인 — Cascade Replication 구조의 EC2 이중 역할:

노드	역할
온프레미스	Primary (Master)
EC2	Slave (to 온프레미스) + Master (to RDS) — 이중 역할 (Relay)
RDS	Slave (to EC2)

MySQL의 `log_slave_updates` 옵션으로 EC2의 이중 역할 가능. 슬라이드 23의 *Relay* 표기가 이걸 의미.

Q15. Split-brain이 뭔가요? — 슬라이드 [42] 박스 ② 관련

"단어 그대로 '분리된 뇌' — 하나의 시스템 안에 master 역할을 하는 노드가 둘 이상 동시에 존재하는 상황입니다. 저희 맥락으로 풀면 — Failover 후 RDS가 새 master로 앱 write를 받고 있는데, 온프레미스 DB가 살아 돌아왔을 때 EC2 격리가 안 되어 있으면, EC2가 온프레미스의 stale write를 replication으로 받아서 RDS로 전파합니다. RDS

한 노드에 *앱의 새 write*와 *EC2 경유 stale write* 두 흐름이 동시에 들어오면서 데이터가 갈라집니다. 슬라이드 [42] 박스 ②의 SQL 세 줄이 바로 이 split-brain을 차단하는 코드입니다."

Q16. Cascade Replication에서 EC2는 RDS에 대한 master 역할도 하지 않나요? — Q15 후속 (날카로운 질문) ★

"맞습니다. Cascade 구조에서 EC2는 *온프레임에 대해서는 slave*, *RDS에 대해서는 master* — 두 역할을 동시에 가집니다. MySQL의 `log_slave_updates` 옵션으로 가능해지고, 슬라이드 23의 *Relay* 표기가 정확히 이걸 의미합니다. 그래서 Failover 시 EC2 격리는 단순한 read-only만으로는 부족합니다. EC2의 *두 역할을 모두 봉인*해야 합니다 — *온프레임 → EC2 경로(slave 역할)*와 *EC2 자체에서 발생하는 write(master 역할의 진입점)* 양쪽 모두 차단해야 합니다. 그래서 SQL이 한 줄이 아니라 세 줄입니다."

💡 정직하게 "맞습니다" 부터 인정. 방어적으로 "아니요, EC2는 slave일 뿐..." 식으로 답하지 말 것.

Q17. SQL 세 줄이 각각 정확히 무엇을 막나요? — Q15 후속

"EC2의 두 역할(slave-to-온프레임 + master-to-RDS) 양쪽을 모두 봉인하는 3중 안전장치입니다.

- **STOP SLAVE** — EC2가 온프레임으로부터 binlog를 끌어오는 것을 차단. (slave 역할 정지)
- **RESET SLAVE ALL** — EC2의 master 정보(온프레임 정보)를 완전히 삭제해서 재연결을 막음. (slave 메타 정보 제거)
- **super_read_only=ON** — EC2에 누가 직접 접속해 write하는 것을 root 권한 포함해서 차단. (master 역할의 진입 차단)

한 줄이라도 빠지면 차단이 헐거워집니다. 예를 들어 `super_read_only`만 켜고 `STOP SLAVE`를 빼면, 온프레임 살아 돌아왔을 때 EC2가 그 write를 계속 받아서 RDS로 전파해버립니다."

Q18. GTID Cascade 복제는 본인이 설계했나요?

"GTID와 Cascade 자체는 DB 담당 팀원이 설계했습니다. 슬라이드 26-29에서 다뤘어요. 제 영역은 그 위에서 동작하는 Pipeline — Failover 시 EC2를 격리하고 RDS를 승격시키는 SQL 호출 순서, Failback 시 Jenkins가 양쪽 네트워크에 닿을 수 있다는 점을 활용해 RDS에서 직접 `mysqldump`를 떠 온프레임으로 복원하는 경로 — 그 흐름을 설계했습니다."

6. 시나리오 / 코드 구조

Q19. 시나리오 1은 왜 Terraform 호출이 없나요?

"시나리오 1은 새 jar만 배포하는 흐름이라 인프라 변경이 0입니다. 그런데도 `terraform apply`를 호출하면 *변경 사항 없음*이라는 plan 결과만 나오는데 plan 시간은 그대로 소비됩니다. 영향 범위에 맞춰서 도구를 호출하는 게 합리적이라고 봤습니다."

Q19-A. 시나리오 분기를 별도 파일로 나누지 않고 단일 Jenkinsfile에서 처리한 이유는? ★

🗨️ [최종 대본 슬라이드 [41]에서 잘린 결정 이유 — Q&A 백업] "안전장치 코드, 환경 변수, *Tailscale* 연결 정보가 시나리오 간에 겹치는 부분이 많았기 때문입니다. 별도 파일로 나누면 그 공통 부분이 각 파일에 사본으로 들어가서, 한 곳을 고치면 다른 곳도 같이 고쳐야 하는 부담이 생깁니다. '단일 파일 + 분기 방식'이 코드 중복 회피와 일관성 측면에서 저희 환경에 더 적합하다고 봤습니다."

Q19-B. 왜 webhook 자동 트리거가 아니라 `parameters { choice() }` 드롭다운 방식이었나요? ★

🗨️ [최종 대본 슬라이드 [42]에서 잘린 결정 이유 — Q&A 백업] "Jenkins에는 webhook 트리거, Multibranch Pipeline 같은 자동 방식도 있지만, '1-Click DR'의 정의에서 가장 중요한 게 '사람 결정' 단계라고 봤기 때문입니다. `choice()` 블록은 운영자가 드롭다운에서 시나리오를 고르는 순간 = 사람 판단이 들어가는 단계와 직접 매칭됩니다. webhook 자동 트리거였다면 일시적 장애에도 Failover가 트리거되어 비용-split-brain 위험이 생길 수 있는데, 사람 결정 단계를 명시적으로 끼워넣는 트리거 방식이 그 위험을 차단합니다."

💡 Q11(왜 인간 결정?)의 후속으로 받을 가능성도 있음 — 왜 결정이 필요한가(Q11) → 어떤 메커니즘으로 결정을 받는가(Q19-B).

Q20. 멍등성이 뭔가요?

"멍등성은 같은 *playbook*을 100번 실행해도 100번 모두 결과가 같은 성질입니다. Failover/Failback 빌드는 같은 코드가 시나리오마다 반복 실행되는데, 두 번 실행했을 때 결과가 달라지면 신뢰성이 무너집니다. Ansible의 멍등성이 이걸 보장해주기 때문에 같은 *role*을 안전하게 반복 호출할 수 있습니다."

Q21. site.yml과 webwas.yml은 정확히 어떻게 다른가요?

"*site.yml*은 6 roles 모두 적용 — 온프레임 환경 풀셋업(HAProxy / Tailscale / MySQL / SpringBoot / common / cloudwatch-agent). 시나리오 3(Failback)에서 온프레임 환경을 처음부터 재구축할 때 사용합니다. *webwas.yml*은 3 roles만 — *common*, *springboot*, *cloudwatch-agent*. WEB-WAS 호스트만 대상으로 jar 갱신 + 서비스 재시작. 시나리오 1의 평상시 앱 배포 전용입니다. 두 *playbook*이 같은 *springboot* role을 호출 — 그래서 '같은 코드, 다른 시나리오'의 의미예요."

Q21-A. 같은 role을 두 playbook에서 재사용한 이유는?

🗨️ [최종 대본 슬라이드 [43]에서 잘린 detail — Q&A 백업] "코드 중복을 피하기 위해서입니다. WEB/WAS 셋업 코드를 *webwas.yml*과 *site.yml* 양쪽에 따로 적으면, 한 곳을 고칠 때 다른 곳도 같이 고쳐야 해서 누락 위험이 생깁니다. role 단위로 추출해서 양쪽이 같은 role을 호출하면 한 곳만 고쳐도 양쪽에 자동으로 반영됩니다. '단일 코드 + 환경별 분기' 원칙이 Jenkinsfile뿐 아니라 Ansible에서도 같은 결로 적용된 부분입니다."

Q22. Ansible 말고 Bash script로도 되지 않나요?

"가능하지만 멍등성을 직접 구현해야 하고, 호스트 여러 대 관리 시 ssh 루프를 직접 돌려야 합니다. 변수 관리, 템플릿 처리, 에러 핸들링 모두 직접 — 결국 Ansible을 다시 만들게 됩니다. YAML 한 번 쓰면 끝나는 걸 굳이 안 해도 되죠."

Q23. 멍등성 보장 안 되는 task도 있나요?

"있습니다. shell 모듈로 현재 상태 무시하고 항상 실행하는 task — 이건 Ansible idiom 위반이라 review 단계에서 걸려야 합니다. 우리 프로젝트는 검증 task 외에는 모두 멍등성 모듈을 사용했어요."

Q24. 왜 Spring Boot 단일 jar로 바꿨나요? Nginx + Tomcat도 정상 동작하는 구성인데

"세 가지 이유였어요. 첫째, **단일 jar로 빌드/배포가 간편** — CI/CD 단순화. WAR + Tomcat manager 호출 같은 다단계 절차가 필요 없습니다. 둘째, **임베디드 Tomcat**으로 별도 WAS 설정이 필요 없어요. 의존성과 서버가 jar 하나에 다 들어 있어 self-contained입니다. 셋째, **jar 하나만 갱신하면 되니 자동화 코드도 단순해진다**는 점 — 이게 발표에서 보여드린 'Ansible role 재사용'의 enabler가 됐어요. 한 영역 단순화가 인접 영역까지 단순화한 케이스예요."

7. AI 협업 / 트러블슈팅

Q25. AI 협업으로 해결한 트러블슈팅 사례는?

"세 가지 있습니다. *Tailscale ProxyJump* — 4시간 헤매다가 AI에게 Subnet Router 우회 제안받아 10분 만에 해결. 결과적으로 HAProxy-only Tailscale 아키텍처의 출발점이 됐습니다. *IAM Bootstrap* — terraform init 시 권한이 권한을 만들어야 하는 닭/달걀 상황이었는데, 사전에 분리해둔 admin 계정으로 풀었습니다. *Ansible ansible_user* — Permission Denied 메시지에서 사용자명이 jenkins로 잡히는 부분에서 의문이 출발해서, ansible_user 명시 한 줄로 해결했습니다. 세 사례 모두 *의문은 제가, 진단 가속은 AI, 결정은 다시 제가* 라는 같은 패턴이었습니다."

Q26. AI가 답을 다 줬다면 본인 기여는 뭐예요?

"AI는 가설 후보를 빠르게 제공해줬고, 저는 그걸 검증·판단·채택했어요. 첫 의문을 던진 것도 저였습니다. 같이 결론에 도달한 거지, AI가 혼자 답을 준 건 아니에요."

💡 절대 "AI가 답을 준 게 아니라" 같은 단언형 X. "AI와 함께", "같이" 같은 협업 톤 사용.

Q27. 가장 어려웠던 디버깅은 뭐였나요?

"슬라이드의 세 사례 외에 백업으로 GTID Multi-Node Align 트러블도 있었어요. 슬라이드에는 안 넣었지만 자세히 풀어드릴 수 있습니다."

📖 백업: [dr-project-troubleshooting-narratives.md](#) 의 GTID 백업 narrative 활용

Q28. AI 사용에 대한 본인 입장은?

"AI는 가설을 빠르게 보여주는 도구로 정말 강력했어요. 다만 그 가설이 우리 환경에 맞는지·다른 부작용이 없는지 검증하는 건 결국 사람의 영역이라고 느꼈습니다. 저는 아직 익혀가는 중이고요."

💡 "이게 답이다" 보다 "제가 느낀 건", "익혀가는 중" 같은 겸손 표현.

8. 향후 개선 / 모니터링

Q29. 9개 영역 중 가장 시급한 것 한 가지를 꼽는다면?

"1순위 안에서도 HA(Active-Standby + keepalived) 라고 봅니다. 현재 HAProxy가 단일 인스턴스라 이 노드 자체가 SPOF(Single Point of Failure)입니다. 이게 다운되면 DR 전체 트리거 조건이 되어 비용·복구 시간이 오히려 늘어납니다. 그래서 가장 먼저 해결할 영역으로 봤습니다."

Q29-A. 1순위에 IAM이 들어간 이유는? (보안인데 안정성 컬럼?) ★

📖 [최종 대본 슬라이드 [45]에서 < > 안 detail이 잘린 부분 — Q&A 백업] "1순위 분류 기준이 '사용자에게 직접적인 영향이 얼마나 가는가' 입니다. IAM 권한이 broad한 상태에서 침해 사고가 일어나면 영향 범위가 시스템 전체로 번지는데, 이건 가용성 장애만큼 사용자에게 직접 타격을 줍니다. 그래서 HA·Multi-AZ와 같은 1순위에 묶였습니다 — 영역은 다르지만(가용성 vs 보안), 사용자 영향 기준으로는 같은 우선순위라는 의미입니다."

Q29-B. 2순위 운영 자동화가 정확히 무엇을 자동화하는 건가요? ★★

💡 [최종 대본 슬라이드 [45]에서 < > 안 detail이 잘린 부분 — Q&A 백업] "운영 품질의 자동화입니다. 비용 절감이 아닌, 운영 일관성과 복구 능력 강화가 목적입니다."

- **Monitoring (SLO + error budget):** 임계값 직관 운영을 목표 가용성 기준 알람으로 바꿔 알람 정밀도 향상
- **Secrets (Secret store 통합 + 자동 회전):** tfvars + Ansible Vault에 흩어진 자격증명을 중앙 저장소로 통합 + 자동 회전
- **Backup (RDS retention 30일 + PITR):** 현재 retention 0일을 30일로 + Point-in-Time Recovery 활성화

💡 "비용 절감 영역이 아니라 운영 품질 영역" 이 핵심. "비용" 단어는 의식적으로 회피.

Q29-C. 3순위 최적화에서 비용 절감 효과가 큰 항목은?

💡 [최종 대본 슬라이드 [45]에서 < > 안 detail이 잘린 부분 — Q&A 백업] "Tailscale 호스트 수입입니다. 현재는 무료 tier로 4대 쓰고 있는데, 유료 tier로 넘어가면 호스트 수가 비용 결정 인자가 됩니다. mesh가 아닌 HAProxy-only Subnet Router 패턴으로 4대 유지하는 게 미래 라이선스 비용을 절감하는 핵심 결정이었습니다 — 슬라이드 [40]의 6→4 framing과 같은 맥락입니다. 그 외 **NAT GW 트래픽**도 VPC Endpoint를 추가하면 인터넷 경로 트래픽을 절감할 수 있습니다."

Q29-D. 비용 영역이라면 왜 Tailscale 라이선스가 1순위가 아닌가요?

💡 [최종 대본 슬라이드 [45]에서 < > 안 detail이 잘린 부분 — Q&A 백업] "분류 기준이 '사용자에게 직접적인 영향' 이기 때문입니다. Tailscale 호스트 수가 늘어도 사용자 서비스는 그대로 동작합니다 — 운영 비용만 증가할 뿐이에요. 반면 HA가 무너지면 즉시 서비스 중단이라 1순위입니다. 비용은 운영 부담에 가까운 영역이라 사용자 직접 영향이 적은 3순위로 분류했습니다."

Q30. Active-Standby + keepalived가 뭔가요?

"HAProxy 두 대를 띄워서 한 대는 Active로 트래픽을 받고 다른 한 대는 Standby로 대기하는 구조입니다. keepalived가 두 대 사이에서 *VIP(Virtual IP)*를 관리하고, Active가 죽으면 VIP를 Standby로 자동 이전해서 무중단 전환합니다. 인프라 도메인의 표준 HA 패턴 중 하나입니다."

Q31. SLO + error budget은 뭔가요?

"현재 모니터링이 임계값 직관 — 예를 들어 CPU 80% 넘으면 알람 — 으로 운영됩니다. 이것을 Service Level Objective(SLO) 기준으로 바꾸는 겁니다. '한 달 안에 99.9% 가용성을 목표로 한다' 같은 목표를 정하면, 거기서 깨질 수 있는 0.1%가 error budget이 됩니다. 임계값보다 실제 사용자 영향 기준이라 알람의 정확도가 올라갑니다."

Q32. Secret store 통합 또는 PITR은 뭔가요?

"Secret store는 인증 정보를 AWS Secrets Manager 같은 중앙 저장소로 통합해서 관리 포인트를 단일화하고 자동 회전을 켤 수 있는 구조입니다. PITR은 Point-in-Time Recovery의 약자로, 백업 retention 안의 임의의 시점으로 RDS를 복구할 수 있는 기능입니다. 둘 다 다음 단계로 학습하면서 적용할 영역입니다."

9. 데이터 정합성 보완 멘트

Q&A 중간에 "데이터가 진짜 일치하는지 확인했나요?" 류 질문이 들어왔을 때 사용.

"데이터 정합성은 시연 영상에서 사용자 측면(페이지 화면)으로 확인했습니다. 추가로 cascade replication을 통해 onprem master → AWS DB EC2 (Relay) → AWS RDS까지 실시간 동기화됨을 직접 SQL로도 검증했습니다. 시간 관계상 시연 영상에는 사용자 측면만 담았습니다."

발표 직전 30초 체크 — Q&A 톤 원칙

- ☐ 단언 회피 — "이게 답이다" / "표준이다" / "AI 시대 엔지니어의" 패턴 X
- ☐ 졸업생/학습자 톤 유지 — "~한 사례라고 생각합니다", "제가 익혀가고 있는"
- ☐ 모르는 건 모른다고 — 팀원 영역은 referrer ("슬라이드 N에서 다뤘습니다")
- ☐ Q9(까칠한 톤)·Q16(EC2 master 역할) 들어왔을 때 — 방어적으로 답하지 말고, 인정부터 + framing
- ☐ AI 협업 — "AI가 답을 준 게 아니라" 단언형 X. "같이", "AI와 함께" 협업 톤

우선순위 — 외워둘 답변 11종

발표 시간 빠듯하면 이것만이라도:

- Q0-A** (CI/CD가 뭔가요?) — 가장 원초적. CI=Continuous Integration, CD=Delivery/Deployment. 이 프로젝트는 *Delivery*에 가까움
- Q0-B** ★★ (본인 파트 정확히 무엇 담당?) — 다섯 가지 역할 + 한 문장 요약 ('세 도구를 1-Click DR로 묶는 일')
- Q0-D** ★★ (어떤 도구로 구현?) — 다섯 가지 도구 (Jenkins/Ansible/Terraform/Tailscale/Git) + 비유 (지휘자·연주자·무대)
- Q9** (Terraform 직접 안 짰으면 본인 기여가?) — Pre-Sales 톤 정수
- Q15-Q17** (Split-brain 3종) — 가장 깊은 기술 질문 대응
- Q11** (1-Click인데 왜 인간이 결정?) — 1-Click DR 정의 정수
- Q26** (AI가 답을 다 줬다면?) — 본인 기여 framing
- Q29** (9개 영역 중 가장 시급한 것) — 시스템 한계 인지 보여주기
- Q5-A** 🗯️ (HAProxy-only 처음부터 이 구조였나?) — *대본에서 잘린 origin story*. 트러블슈팅이 더 좋은 설계로 이어진 narrative
- Q19-A / Q19-B** 🗯️ (왜 단일 Jenkinsfile? / 왜 choice() 트리거?) — *대본에서 잘린 결정 이유*
- Q29-B** 🗯️ (2순위 운영 자동화가 정확히 뭘 자동화?) — *비용 X, 운영 품질 O* 명확히 답변

🗯️ 대본에서 잘려나간 Q&A 백업 — 빠른 인덱스

최종 대본이 짧아진 만큼, 아래 detail은 청중이 물어볼 가능성이 더 높아짐:

Q	잘린 슬라이드	핵심 내용
Q2-A	[5] Pilot Light	SpringBoot ASG 0→2 <i>깨어나는 자원</i> + Jenkins/HAProxy는 평소 가동
Q2-B	[6] 네트워크	온프레임 192.168.20.0/24 + AWS VPC 10.0.0.0/16
Q5-A	[40] Tailscale	mesh → ProxyJump 트러블슈팅 → HAProxy-only 발견 origin story
Q19-A	[41] 시나리오	단일 Jenkinsfile 채택 이유 (안전장치/Tailscale 정보 공유)
Q19-B	[42] Jenkinsfile	choice() 트리거 채택 이유 (사람 결정 명시)

Q	잘린 슬라이드	핵심 내용
Q21-A	[43] Ansible	같은 role 재사용 이유 (코드 중복 회피)
Q29-A	[45] 한계 1순위	IAM이 1순위에 들어간 이유 (사용자 영향 기준)
Q29-B	[45] 한계 2순위	2순위 = 운영 품질 자동화 (비용 절감 ✖) — 3개 항목 detail
Q29-C	[45] 한계 3순위	3순위 비용 절감 큰 항목 = Tailscale 호스트 수 + NAT GW
Q29-D	[45] 한계 분류	비용 영역인데 왜 1순위가 아닌가 — 사용자 영향 기준